NIT ACADEMY

TFTP, UDP, tcpdump & iptables

Lessons Learned + Hands-On Linux Firewall Troubleshooting Lab

Scenario: A Linux XCP-ng hypervisor could send a TFTP request to a Windows Tftpd64 server, but the download repeatedly timed out. Packet captures, firewall counters, and a temporary test rule isolated the fault to the Linux firewall.

System	Address	Function
XCP-ng: nit-xcp01	192.168.1.236	Linux TFTP client; interface xenbr0
Windows 11	192.168.1.142	Tftpd64 server; UDP port 69
iDRAC7	192.168.1.139	Separate management controller; not involved in the successful client test

1 | Learning outcomes

By completing this lab, students can: explain UDP and TFTP transfer ports; distinguish a packet arriving at a NIC from reaching an application; read iptables chains, policies, counters, and rule order; capture UDP and ICMP using tcpdump; interpret administrative-prohibited ICMP; understand connection-tracking helpers; prove a firewall hypothesis using a narrowly scoped test; and restore original rules afterward.

2 | TFTP and UDP: what happens on the wire?

```
Linux client                    Windows TFTP server
192.168.1.236                   192.168.1.142
  UDP ephemeral:38275  -------->  UDP 69       (RRQ)
  UDP ephemeral:38275  <--------  UDP 64211    (DATA/OACK)
  UDP ephemeral:38275  -------->  UDP 64211    (ACK)
       ^ missing ACK when Linux firewall rejected replies
```

Important: TFTP uses UDP, not TCP. Port 69 receives the initial read request (RRQ); the server normally transfers data from a newly selected UDP transfer identifier (TID), typically an ephemeral port. Therefore, allowing only UDP destination port 69 on the client does not automatically allow the server's return packets.

UDP vs TCP: TCP is connection-oriented and tracks acknowledgments, retransmissions, and ordered byte streams in the transport protocol. UDP is connectionless; TFTP implements its own block acknowledgments and retransmissions at the application layer. The curl message Connected for a UDP URL does not prove a completed bidirectional transfer.

3 | Original failure and first observations

```
curl -v --max-time 15 \
  tftp://192.168.1.142/tftp-test.txt \
  -o /tmp/tftp-test.txt
# curl: (28) Operation timed out ... 0 bytes received
```

The Windows Tftpd64 log showed incoming read requests and repeated retransmissions without ACKs. This indicated that the server received the request, but did not prove where the return path failed.

4 | Inspect the Linux firewall

```
iptables -L INPUT -n -v
iptables -L RH-Firewall-1-INPUT -n -v
iptables -L RH-Firewall-1-INPUT -n -v --line-numbers
```

Option	Meaning
-L	List rules in a chain
-n	Show numeric IP addresses and ports (no name resolution)
-v	Verbose view, including packet/byte counters
--line-numbers	Show positions for interpreting insertion order

The INPUT chain jumped to RH-Firewall-1-INPUT. Within that chain, rule 4 allowed RELATED,ESTABLISHED traffic; the last rule (initially number 11) was REJECT all ... reject-with icmp-host-prohibited. A permissive INPUT policy does not override a REJECT rule reached in a referenced chain.

Check counters without altering rules

```
iptables -L RH-Firewall-1-INPUT -n -v | grep REJECT
```

Recorded counter values rose from 31,218 to 32,328 (1,110 more packets). This was a clue, not definitive proof: counters include unrelated traffic. A counter rising during a test is correlation unless the matching packets are isolated.

5 | Establish the packet path using tcpdump

```
ip route get 192.168.1.142
# 192.168.1.142 dev xenbr0 src 192.168.1.236

command -v tcpdump
# /usr/sbin/tcpdump

timeout 30 tcpdump -ni xenbr0 -vv \
  "host 192.168.1.142 and udp"
```

The capture showed a Linux RRQ from 192.168.1.236:38275 to 192.168.1.142:69, followed by Windows UDP replies from 192.168.1.142:64211 to 192.168.1.236:38275. Repeated server replies without a client ACK were visible. This established that response packets reached the Linux network interface, not that the application received them.

```
timeout 20 tcpdump -ni xenbr0 \
  "icmp and host 192.168.1.142"
# 192.168.1.236 > 192.168.1.142:
# ICMP host 192.168.1.236 unreachable - admin prohibited
```

Interpretation: Linux generated administrative-prohibited ICMP messages toward the Windows sender during the TFTP test. Together with the REJECT rule and subsequent successful controlled test, this identified the firewall as the cause. A UDP checksum warning on locally transmitted packets may reflect NIC checksum offload; it was not the demonstrated cause here.

6 | Connection tracking: why RELATED did not save the transfer

```
lsmod | grep -E 'nf_conntrack_tftp|ip_conntrack_tftp'
modinfo nf_conntrack_tftp
# Module available in /lib/modules/...
modprobe nf_conntrack_tftp
lsmod | grep nf_conntrack_tftp
```

The helper was initially absent from the loaded-module list, but modinfo found it. Loading it succeeded; however, the TFTP transfer still timed out. Key distinction: a helper module being loaded does not necessarily mean it is assigned to the flow. Depending on kernel and conntrack configuration, explicit helper assignment may be required. We did not prove whether helper assignment was active, and did not make it part of the final fix.

7 | Controlled test rule and confirmed outcome

Production caution: The command below was used in this real incident, but should be performed only in an authorized lab or controlled maintenance window. It permits all UDP ports from one source IP to one destination IP, so it is broader than a permanent production rule should be.

```
iptables -I RH-Firewall-1-INPUT 11 \
  -i xenbr0 -p udp \
  -s 192.168.1.142 -d 192.168.1.236 \
  -j ACCEPT

iptables -L RH-Firewall-1-INPUT -n -v --line-numbers | tail -3
```

The output confirmed that the new ACCEPT rule was inserted immediately before REJECT. Repeating the same curl request downloaded 36 bytes. Reading the file returned the expected text:

```
cat /tmp/tftp-test.txt
# Hello from NIT Academy TFTP Server
```

Conclusion: With the Windows server and Linux client unchanged, the transfer succeeded after the targeted firewall allowance. This is the decisive intervention supporting the firewall diagnosis.

Remove the temporary rule

```
iptables -D RH-Firewall-1-INPUT \
  -i xenbr0 -p udp \
  -s 192.168.1.142 -d 192.168.1.236 \
  -j ACCEPT

iptables -L RH-Firewall-1-INPUT -n -v --line-numbers | tail -3
```

Recordkeeping: The removal command was provided in the conversation, but a successful cleanup output was not yet shown. Verify its removal before treating the production change as closed. Do not blindly delete rules on a shared hypervisor.

8 | Student practical lab: reproduce, observe, explain

Use two isolated training VMs, not the production XCP-ng host. Suggested topology: Linux VM-A (client) 192.168.56.10/24; Linux VM-B (server) 192.168.56.20/24; private virtual switch, no public network access. Run commands with sudo where needed. Packages and service names vary by distribution; example below assumes a RHEL/Rocky-like environment with a TFTP server and tcpdump available.

Lab A - Establish a baseline

On VM-B, configure a TFTP service rooted at its configured TFTP directory (commonly /var/lib/tftpboot), place a readable file named tftp-test.txt containing the message below, and verify the server is listening on UDP 69. Consult the installed service configuration; do not assume a specific daemon or firewall is present.

```
echo "Hello from NIT Academy TFTP Server" | \
  sudo tee /var/lib/tftpboot/tftp-test.txt

sudo ss -lunp | grep ":69"
```

On VM-A, test the download and inspect the file. A successful baseline is essential before deliberately blocking anything.

```
curl -v --max-time 15 \
  tftp://192.168.56.20/tftp-test.txt \
  -o /tmp/tftp-test.txt
cat /tmp/tftp-test.txt
```

Lab B - Observe the UDP port change

On VM-A, identify the outgoing interface, then capture traffic while repeating the download from another terminal. Note the source and destination ports in the RRQ and server replies.

```
ip route get 192.168.56.20
sudo timeout 25 tcpdump -ni <CLIENT_INTERFACE> \
  "host 192.168.56.20 and udp"
```

Expected observation: the client sends an RRQ to UDP 69; the server replies from a different UDP source port. Replace <CLIENT_INTERFACE> with the actual interface reported by ip route.

Lab C - Reproduce a reject safely

For this exercise, use a dedicated disposable VM with local console access and a known rollback/snapshot. Avoid changing INPUT policy or broad firewall chains. The example creates a separate diagnostic chain and attaches it only to UDP packets from VM-B destined for VM-A. It deliberately blocks all matching UDP replies, then removes the test rule. If your environment is managed by firewalld/nftables, adapt the exercise rather than mixing persistent managers.

```
sudo iptables -N NIT_TFTP_LAB
sudo iptables -A NIT_TFTP_LAB \
  -j REJECT --reject-with icmp-host-prohibited
sudo iptables -I INPUT 1 -p udp \
  -s 192.168.56.20 -d 192.168.56.10 \
  -j NIT_TFTP_LAB

sudo iptables -L NIT_TFTP_LAB -n -v
# In another terminal: retry the curl download
```

The curl transfer should time out if this INPUT hook is active and the server response matches. Watch the counter for the dedicated chain, which is a stronger test than watching a shared catch-all REJECT counter.

```
watch -n 1 "sudo iptables -L NIT_TFTP_LAB -n -v"
```

In a separate terminal, capture ICMP while re-running curl:

```
sudo timeout 20 tcpdump -ni <CLIENT_INTERFACE> \
  "icmp and host 192.168.56.20"
```

Expected: a Linux-generated administrative-prohibited ICMP message to the server, provided the packet is not rejected earlier by another rule and the capture interface is correct.

Lab D - Restore service and validate

Remove the dedicated test hook and chain in this order. Then repeat curl and confirm successful download. Do not leave the reject rule installed.

```
sudo iptables -D INPUT -p udp \
  -s 192.168.56.20 -d 192.168.56.10 \
  -j NIT_TFTP_LAB
sudo iptables -F NIT_TFTP_LAB
sudo iptables -X NIT_TFTP_LAB

curl -v --max-time 15 \
  tftp://192.168.56.20/tftp-test.txt \
  -o /tmp/tftp-test.txt
cat /tmp/tftp-test.txt
```

Lab safety: If the lab host already has restrictive INPUT rules, adjust the baseline first. Some Linux distributions use nftables backends or firewalld; verify the active firewall manager before running these commands. Always retain VM console access and take a snapshot. Do not run this lab on a production hypervisor.

9 | Student questions and assessment

Question	Expected answer
Why can a TFTP RRQ succeed but download fail?	The reply may originate from a different UDP port and be rejected by the client firewall.
What does a rising REJECT counter prove?	Some packets matched the rule; it does not identify which flow without further evidence.
Why capture UDP and ICMP separately?	UDP shows request/reply traffic; ICMP can reveal an administrative firewall rejection.
Does tcpdump seeing a packet prove curl received it?	No. Capture can observe packets before local firewall/application processing.
Did loading nf_conntrack_tftp fix the incident?	No. It loaded, but the download still timed out.
What proved the diagnosis?	A controlled allow rule was inserted; the same file transfer then succeeded.
What must be done after the test?	Remove temporary rules, verify restored rules, and document the outcome.

10 | One-page troubleshooting checklist

☐ Confirm the server has the requested file and TFTP service is running.

☐ Run curl with -v and a bounded timeout; capture exact error.

☐ Inspect INPUT chain, referenced chains, ordering, and counters.

☐ Use ip route get to identify the interface and source IP.

☐ Use tcpdump to confirm RRQ to UDP 69 and response from a dynamic port.

☐ Look for ICMP administrative-prohibited messages.

☐ Check connection tracking and helper status without assuming loaded means active.

☐ On an isolated lab, use a narrowly scoped, reversible test rule.

☐ Repeat the same transfer and validate file contents.

☐ Remove temporary rules and verify cleanup; record what was proven versus inferred.

Key takeaway: Successful delivery to a network interface is not the same as successful delivery to an application. Packet captures + firewall-rule order + counters + a controlled test establish causality much more reliably than counters alone.